Rituben Piyushbhai Patel

main

New Assignment

Document Details

Submission ID
trn:oid:::12112:137509320

Submission Date
May 3, 2026, 6:46 PM GMT+5:30

Download Date
May 3, 2026, 6:58 PM GMT+5:30

File Name
main.pdf

File Size
5.0 MB

50 Pages

10,008 Words

55,128 Characters

77% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

111 AI-generated only 77%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Agentic Cloud Cluster

A Go Distributed Cluster Managing Framework with PPO-based Scheduling

A Project Report

Submitted by

Sarthak Siddhpura

Enrollment Number: AU2240041

in partial fulfillment for the award of the degree

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Internal Faculty Mentor

Prof. Sanjay Chaudhary

Ahmedabad University

School of Engineering and Applied Science (SEAS)

Ahmedabad University

Ahmedabad, Gujarat

May, 2026

DECLARATION

I hereby declare that the project entitled "**Agentic Cloud Cluster: A Go Distributed Cluster Managing Framework with PPO-based Scheduling**" submitted for the B. Tech. degree is my original work and the project has not formed the basis for the award of any other degree, diploma, fellowship, or any other similar title.

Signature of Student

Date:

Place: Ahmedabad

CERTIFICATE

This is to certify that the project titled "**Agentic Cloud Cluster: A Go Distributed Cluster Managing Framework with PPO-based Scheduling**" is the bona fide work carried out by **Sarthak Siddhpura**, Enrollment Number **AU2240041**, a student of B. Tech. in Computer Science and Engineering at the School of Engineering and Applied Science, Ahmedabad University, during the academic year 2025–2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology.

This project was done under the supervision of the faculty mentor **Prof. Sanjay Chaudhary**.

Signature of Faculty Mentor

Date:

Place: Ahmedabad

Abstract

Agentic Cloud Cluster is a distributed task execution and scheduling framework built in Go. The system manages a master node, multiple worker nodes, Docker-based task execution, persistent task state, worker heartbeats, task retries, and a scheduler interface that can switch between baseline and learned policies. The main contribution of this project is not only that the cluster runs tasks, but that scheduling is treated as a learning problem. Instead of permanently relying on a fixed rule such as Round-Robin or a manually tuned heuristic, the framework integrates a Proximal Policy Optimization (PPO) scheduler trained on cluster traces and deployed behind a safe Go scheduler boundary.

The report explains the project from the ground up. It first motivates why distributed clusters need scheduling decisions, then explains why reinforcement learning is suitable when the best decision depends on changing resource state. The methodology moves step by step from tabular Q-learning, to Deep Q-Networks, and finally to PPO. The PPO policy observes task requirements and worker resource features, chooses a worker, receives a shaped reward, and improves using clipped policy-gradient updates. The implementation uses Go for the master-worker control plane and Python/PyTorch for the PPO service.

The final system was evaluated through offline trace replay and end-to-end cluster campaigns. The PPO training curve shows convergence through decreasing policy and value losses, while benchmark plots compare PPO against baseline schedulers across resource and workload patterns. The results show that the learned scheduler is most useful when scheduling pressure is high, such as burst and overload scenarios, while simple baselines can remain competitive in light-load cases. This makes the project a practical study of where reinforcement learning helps in systems work and where conservative fallback design is still necessary.

Table of Contents

List of Figures

List of Tables

Gantt Chart

Table 0.0.1: Project timeline

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17
Problem study and literature review	X	X	X														
Go master-worker framework		X	X	X	X												
Worker execution, heartbeats, and persistence				X	X	X	X	X									
Baseline schedulers and scheduler interface						X	X	X	X								
PPO training pipeline and trace replay								X	X	X	X	X					
PPO integration with Go control plane											X	X	X				
Benchmark campaigns and result analysis													X	X	X	X	
Report writing and final polishing																	X

The timeline now spans 17 weeks. The early weeks established the distributed cluster baseline, the middle weeks focused on PPO training and integration, and week 17 was reserved exclusively for documentation and polishing.

Chapter 1

Introduction

Modern computing workloads are rarely executed on a single machine. A useful system must accept many tasks, understand what resources each task needs, select a suitable worker, run the task safely, and store the result. This becomes harder when workers are heterogeneous. One worker may have more CPU, another may have more memory, and the best choice can change every few seconds as tasks start and finish.

The project developed in this work is called **Agentic Cloud Cluster**. It is a Go-based distributed cluster managing framework that schedules Docker-based tasks across worker nodes. The focus of the project is the cluster framework and the PPO-based scheduler. Other surrounding utilities are treated only as supporting parts of the system.

1.1 | Project Definition

The project can be defined as follows:

> To build a distributed task execution framework in Go and improve task placement decisions using a reinforcement learning scheduler based on Proximal Policy Optimization.

The system has a master node and multiple worker nodes. The master accepts tasks, tracks workers, stores task state, and asks a scheduler to select a worker. The worker executes the task using Docker containers and reports status back to the master. The scheduling algorithm can be changed without changing the worker execution logic.

1.2 | Motivation

The first scheduling solution most systems use is a simple rule. Round-Robin is easy to understand: send one task to worker 1, the next to worker 2, and so on. This is useful as a baseline, but it ignores whether the worker has enough resources, whether it is already busy, and whether the task is likely to miss its deadline.

Heuristic schedulers improve this by adding rules. For example, a scheduler can prefer the worker with the most available CPU or the lowest estimated risk. However, fixed rules are difficult to tune because the right trade-off changes with workload type. A CPU-heavy task, a memory-heavy task, and a burst of short tasks do not need the same decision rule.

This is why reinforcement learning was explored. In reinforcement learning, an agent learns by taking actions and receiving rewards [1]. For cluster scheduling, the state is the current cluster condition, the action is the selected worker, and the reward measures whether the placement was useful. This matches the nature of scheduling: every placement changes the next state of the cluster.

1.3 | Project Objectives

The objectives of the project were:

- Build a working Go master-worker cluster that can run Docker-based tasks.
- Maintain worker registration, heartbeat, resource tracking, task lifecycle, and result storage.
- Provide a clean scheduling boundary so different scheduling algorithms can be compared.
- Implement baseline scheduling and PPO-based scheduling.
- Train PPO using realistic cluster traces and a reward function aligned with feasibility, queue pressure, tail pressure, and load balance.
- Evaluate the system with repeatable campaigns and explain the results honestly.

1.4 | Scope

The core scope is the distributed cluster and PPO scheduler. The report intentionally does not focus on auxiliary interfaces or monitoring tools. They may exist as supporting utilities in the repository, but the technical contribution is the Go control plane, worker execution layer, and reinforcement learning scheduler.

1.5 | Main Contributions

The main contributions are:

1. A Go-based distributed task execution framework with master and worker nodes.
2. A scheduler interface that supports simple baselines and PPO scheduling.

3. A PPO service that receives scheduling requests over gRPC and returns a worker decision.
4. A trace replay training environment with lifecycle-based resource accounting.
5. A reward design that penalizes infeasible placement, queue pressure, tail pressure, and imbalance.
6. A benchmark setup for comparing scheduling behavior across load patterns.

Chapter 2

Literature Survey

2.1 | Related Work

2.1.1 | Distributed Task Scheduling

Distributed systems split work across multiple machines. In a cluster, the scheduler decides where each task should run. This decision affects completion time, resource utilization, fairness, and failure recovery. A scheduler must also react to imperfect information because worker state changes while tasks are being submitted.

2.1.2 | Reinforcement Learning for Scheduling

Reinforcement learning studies how an agent can learn to make decisions by interacting with an environment and receiving reward signals [1]. A standard RL setup has:

- **State**: what the agent observes.
- **Action**: what the agent can choose.
- **Reward**: feedback after the action.
- **Policy**: the rule or model used to select actions.

For this project, the mapping is direct. The state is the current task and workers. The action is the chosen worker. The reward measures placement quality. The policy is the scheduler.

2.1.3 | Q-Learning and Q-Tables

Q-learning is a classic reinforcement learning algorithm introduced by Watkins and Dayan [2]. It learns a value called $Q(s, a)$, meaning the expected long-term reward of taking action a in state s. In a small environment, these values can be stored in a table.

A Q-table is attractive because it is simple. If there are only a few states and actions, the table can store every state-action pair. However, cluster scheduling does not have a small state space. CPU, memory, storage, task type, queue depth, worker availability, and worker load can combine into many possible states. A table would either become huge or require aggressive discretization, which loses important details.

2.1.4 | Deep Q-Networks

Deep Q-Networks (DQN) replace the Q-table with a neural network. Mnih et al. showed that deep neural networks can approximate action-value functions from high-dimensional inputs [3]. This solves the storage problem of Q-tables because the model generalizes across similar states.

DQN is a natural next step after Q-tables, but it is still not ideal for this project. DQN estimates a value for each discrete action. In cluster scheduling, the number of candidate workers can change, workers can become infeasible, and invalid actions must be masked. DQN can be adapted, but the implementation becomes less natural when the action set is dynamic and the objective is to improve a stochastic placement policy.

2.1.5 | Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a policy-gradient algorithm proposed by Schulman et al. [4]. Instead of only learning a Q-value, PPO directly learns a policy that maps states to action probabilities. It also learns a value function to estimate future return. The key idea in PPO is to avoid changing the policy too aggressively by using a clipped objective.

PPO was selected because it fits the scheduling problem better than a Q-table or plain DQN:

- It directly learns a worker-selection policy.
- It can use action masks to avoid infeasible workers.
- It supports stable updates through clipping.
- It works naturally with continuous state features such as resource ratios.
- It allows offline training and optional online adaptation.

2.1.6 | Cluster Trace-Driven Evaluation

Training only on random synthetic tasks can make a scheduler unrealistic. The Alibaba cluster trace provides real production cluster workload data and has been released for cluster management research [5]. This project used Alibaba trace replay to expose the PPO scheduler to realistic task resource patterns and arrival behavior.

2.2 | Tools and Technologies

2.2.1 | Docker-based Execution

Container-based execution is a practical way to make tasks portable. Docker packages an application and its dependencies into a container image, which allows the worker node to run workloads in a repeatable environment [6]. In this project, the worker node uses Docker as the execution boundary while the master node remains responsible for placement decisions.

2.2.2 | Go for Cluster Control Plane

Go was selected for the master-worker framework because it is well suited for networked systems. It has built-in support for concurrency through goroutines and channels, a strong standard library, and efficient compiled binaries. The official Go documentation describes the language as an open-source project designed to make programmers more productive [7]. In this project, Go is used for the master server, worker server, task handling, scheduler interface, and worker communication logic.

2.2.3 | gRPC and Protocol Buffers

The master and worker communicate through gRPC. gRPC is a high-performance Remote Procedure Call framework that uses Protocol Buffers for service definitions and structured messages [8]. This makes it suitable for the project because the master needs strongly defined calls such as worker registration, task assignment, heartbeat reporting, log streaming, and PPO worker selection.

2.2.4 | Persistence Layer

Task execution systems need persistence because a task has a lifecycle: created, queued, assigned, running, completed, failed, or retried. MongoDB was used for storing task state, worker metadata, results, and scheduler model records. MongoDB represents records as documents made of field-value pairs [9], which works well for task and worker objects that may evolve during development.

Chapter 3

Methodology

3.1 | System Design

3.1.1 | High-Level Architecture

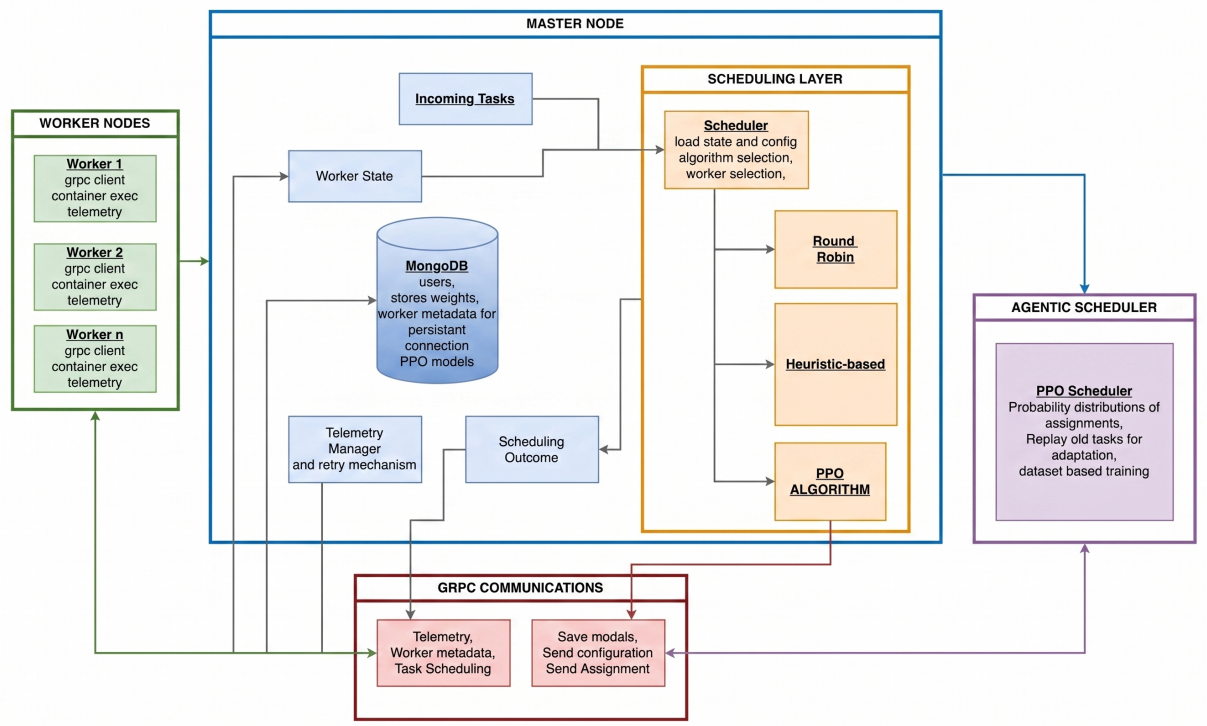

Figure 3.1.1: Overall architecture of Agentic Cloud Cluster

Explanation of Figure 3.1.1: The system is arranged around a master-worker model. The master node receives task submissions, stores metadata, tracks workers, and calls the scheduler. Worker nodes run the assigned Docker tasks and report status back to the master. The PPO scheduler is separated as a Python service so that the Go control plane stays stable while the learning model can evolve independently.

3.1.2 | Go Distributed Cluster Architecture

The distributed cluster is implemented as a control plane in Go. The design follows a simple rule: the master makes coordination decisions, and workers execute tasks. This keeps the system easy to reason about. A worker does not need to know about other workers. It only needs to accept assignments from the master, execute the container, and report health and task status.

At runtime, the cluster has four main layers:

1. **Client and API layer**: receives task submission and management commands.
2. **Master control plane**: validates requests, stores task state, tracks workers, and selects a scheduler.
3. **Scheduler layer**: chooses a worker using Round-Robin, RTS, or PPO.
4. **Worker execution layer**: runs Docker containers and reports results.

The master and workers communicate using gRPC. The request and response formats are defined in Protocol Buffer files, which keeps communication strict and typed. The master also uses persistent storage for task records, worker metadata, task attempts, and result metadata. The important design point is that scheduling, execution, and persistence are separate modules. This made the project easier to debug because a scheduler bug did not require rewriting the worker executor, and a worker execution bug did not require changing the PPO model.

3.1.3 | Master Node

The master node is the coordinator. Its responsibilities are:

- Accept task submissions.
- Maintain worker registry and worker health.
- Track available CPU, memory, and storage.
- Call the active scheduler for each task.
- Send task assignments to workers.
- Store task state, attempts, and results.
- Requeue tasks when workers fail or resources are unavailable.

Internally, the master has a few important modules:

Table 3.1.1: Master node modules

Module	Role
Server layer	Exposes gRPC calls used by workers for registration, heartbeat, task status, and result reporting.
Task handling	Creates tasks, tracks state transitions, queues tasks when workers are not available, and records attempts.
Worker registry	Maintains active worker IDs, addresses, resource capacities, and current availability.
Scheduler package	Provides a common interface for Round-Robin, RTS, and PPO scheduling.
Persistence layer	Stores workers, tasks, assignments, attempts, and results.
Telemetry manager	Maintains recent worker health and resource state for scheduling and operator inspection.

The scheduler boundary is especially important. The master does not need to know the internal algorithm. It only asks: given this task and these workers, which worker should receive the task? The scheduler returns a worker ID, and the master validates that the worker still exists and still has enough resources before dispatching the task.

3.1.4 | Worker Node

The worker node is responsible for executing tasks. Each worker exposes a gRPC server, accepts assignments from the master, starts a Docker container, tracks execution, streams logs, and reports final status. The worker also sends heartbeat data so the master can maintain a live view of cluster resources.

The worker has three main responsibilities:

- **Registration and heartbeat**: once registered, the worker periodically sends resource and status updates to the master.
- **Container execution**: the worker pulls or uses the requested Docker image, creates the container with the requested resource limits, runs it, and observes the exit code.
- **Result reporting**: after the container exits, the worker reports success or failure and uploads output files if the task produced any.

This makes the worker mostly stateless from a cluster point of view. If a worker disappears, the master detects the missed heartbeats and can requeue tasks. The worker does not coordinate with other workers, which avoids complex distributed locking.

3.1.5 | Control Flow in the Go Cluster

The normal task flow is:

1. A task is submitted with image name, command, and resource needs.

2. The master stores the task in created or queued state.
3. The master builds a live view of feasible workers.
4. The active scheduler selects a worker.
5. The master sends the task to that worker through gRPC.
6. The worker runs the Docker container and streams or records logs.
7. The worker reports completion.
8. The master stores the result and releases the worker resources.

The system also supports recovery behavior. If a worker becomes unavailable, the master marks the current attempt as lost and can requeue the logical task. This separation between a logical task and an execution attempt is important because a task may need more than one attempt before it reaches a final state.

3.1.6 | Resource Accounting

Scheduling depends on resource accounting. Each worker advertises total CPU, memory, and storage. The master tracks available resources by subtracting the resources reserved for currently assigned tasks. Once a task completes, those resources are released. This accounting is used both by the simple schedulers and by PPO's action mask.

The design avoids sending tasks to workers that cannot fit them. Even if PPO predicts a worker, the Go master still validates the choice before assignment. This protects the cluster from stale model output or stale worker state.

3.1.7 | Task Lifecycle

Tasks enter the master queue first and remain queued until at least one worker can satisfy the resource constraints. Once a feasible worker is available, the scheduler selects a target and the master creates an execution attempt. If dispatch fails due to stale state or connectivity problems, that attempt is marked lost and the logical task is returned to the queue for another scheduling cycle.

After execution starts, the worker reports completion status back to the master. Successful runs are stored as completed. Failed runs are classified as recoverable or non-recoverable; recoverable failures are retried through requeueing, while non-recoverable failures close the task as failed.

3.1.8 | Task Execution Flow

The concrete execution path starts at the API, where the task is validated and persisted, then moved into the master queue. The queue processor builds a fresh worker snapshot, invokes the active scheduler, and verifies the selected worker is still feasible before reserving resources and dispatching an `AssignTask` call.

If the worker acknowledges and starts the container, the attempt continues until completion and the master persists the final result while releasing reserved resources. If assignment or execution fails due to transient issues (for example worker loss), the attempt is marked lost and the logical task is requeued for rescheduling. This attempt-level recovery is what enables safe retries without losing task-level traceability.

3.1.9 | Scheduler Interface

The scheduler receives:

- task CPU requirement,
- task memory requirement,
- task storage requirement,
- task type,
- worker capacity,
- worker availability,
- worker activity status.

It returns a worker ID. This simple interface allowed the project to start with baselines and later add PPO. The PPO scheduler calls a Python service over gRPC. If the PPO service is unavailable or returns an unsuitable worker, the Go scheduler falls back to a safer scheduler. This was a deliberate design decision because a learning model should improve scheduling, not make the whole cluster fragile.

3.1.10 | PPO Deployment Design

The PPO deployment supports three modes:

Table 3.1.2: PPO deployment modes

Mode	Meaning
Active	PPO makes the scheduling decision, with fallback on failure.
Shadow	PPO is queried for comparison, but the fallback scheduler is used for the actual placement.
Fallback	PPO is bypassed and the fallback scheduler is used directly.

This design was chosen because model deployment in systems work must be conservative. The system can test PPO decisions without immediately trusting them in production paths.

3.2 | Why Reinforcement Learning for Scheduling

Scheduling is a repeated decision problem. Each task placement affects the future state of the cluster. If a large task is placed on the only high-memory worker, a later memory-heavy task may fail or wait. If all tasks are sent evenly without checking resource demand, a weak worker may become overloaded. This is why the scheduler should learn from consequences, not only follow a fixed rule.

In reinforcement learning terms:

- **Agent**: the scheduler.
- **Environment**: the cluster and incoming task stream.
- **State**: task features and worker resource features.
- **Action**: select one worker.
- **Reward**: score the placement based on feasibility, delay risk, and balance.

3.3 | Starting Point: Q-Tables

The simplest RL idea is a Q-table. It stores a value for every state-action pair:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Here, s is the current state, a is the selected action, r is the reward, s' is the next state, α is the learning rate, and γ discounts future reward.

For a small example, this works nicely. Suppose there are only two workers and three task types. A table can store the value of placing each task type on each worker. But the real cluster state is much larger. A worker is not just “free” or “busy”. It has CPU, memory, storage, current load, and active status. A task also has CPU, memory, storage, task type, and SLA multiplier. If each feature is discretized into even a few buckets, the number of table rows grows very quickly.

Decision: Q-tables were useful for understanding the RL formulation, but not suitable for implementation because the state space is too large and continuous.

3.4 | Next Step: Deep Q-Networks

DQN solves the table-size problem by using a neural network to approximate Q-values:

$$Q(s,a;\theta) \approx \text{expected future reward}$$

This is better because the network can generalize. If it has seen a worker at 60 percent memory usage, it can still make a reasonable prediction for 62 percent memory usage.

However, DQN has limitations for this project:

- The action set depends on the number of workers.
- Some workers are invalid for a task because they do not have enough resources.
- The scheduler needs a clean way to mask invalid actions.
- We wanted a direct policy over workers, not only a value estimate.
- Stable DQN training usually requires replay buffers and target networks, which adds complexity when outcomes are delayed.

Decision: DQN was a logical next idea after Q-tables, but PPO was selected because policy-gradient learning matches the worker-selection action more naturally.

3.5 | Final Choice: PPO

PPO directly learns a policy $\pi_\theta(a|s)$, which means the probability of choosing worker a in state s. It also learns a value function $V_\theta(s)$, which estimates how good the current state is.

The main PPO objective used is the clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

where:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta old}(a_t|s_t)}$$

$\hat{A}_t$ is the advantage estimate. It tells whether an action was better or worse than expected. The clipping term prevents the new policy from moving too far away from the old policy in one update. In this project, this stability was important because bad scheduler updates can directly affect task placement.

3.6 | PPO Architecture

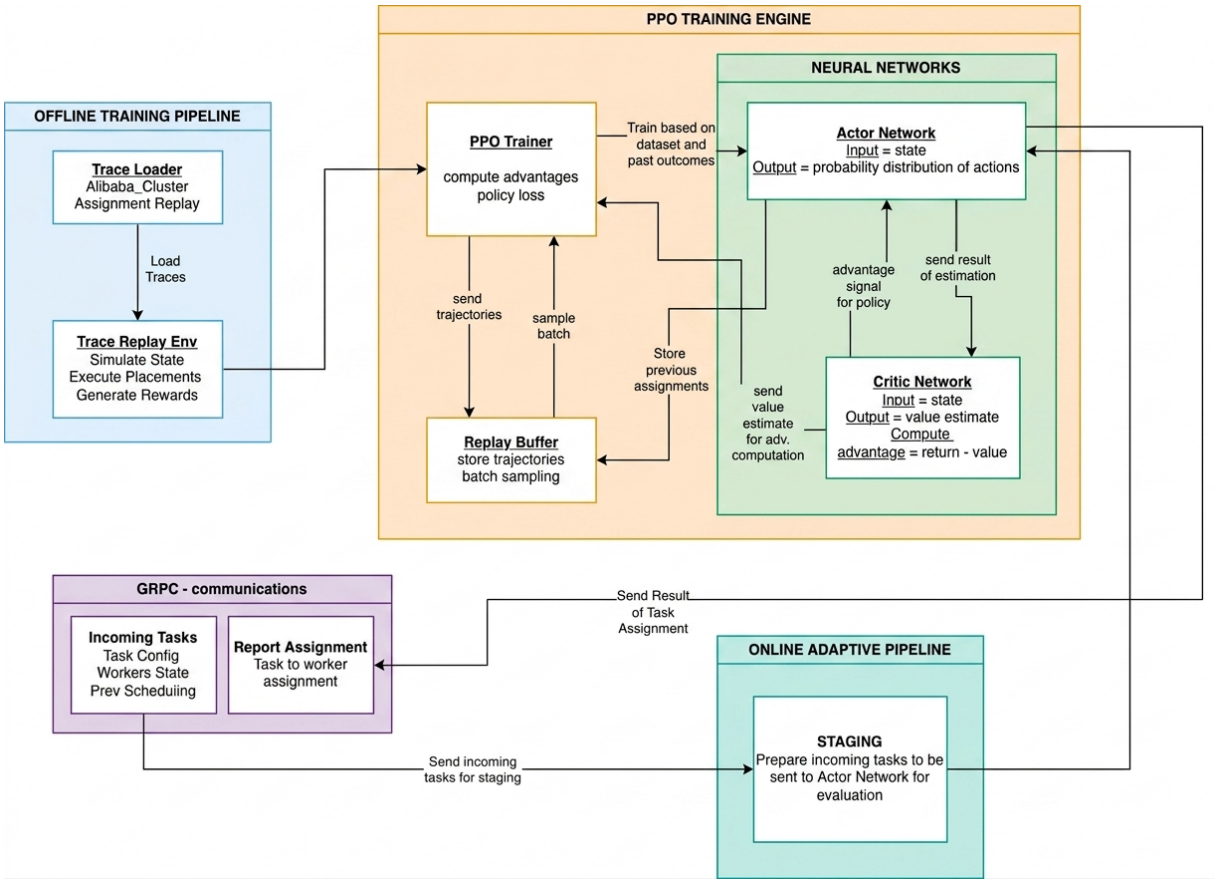

Figure 3.6.1: PPO scheduler architecture

Explanation of Figure 3.6.1: The PPO scheduler receives task features and worker features. These are encoded into pairwise task-worker inputs. The neural network produces policy logits for worker selection and a value estimate for the state. The policy head decides which worker should receive the task. The value head helps calculate advantages during training. The Go master calls the Python PPO service through gRPC, while the Python service loads the trained PyTorch checkpoint.

3.7 | Actor and Critic Network

The PPO model is an actor-critic network. Both the actor and critic share the same pairwise encoder. For each scheduling request, the task vector is repeated once for every candidate worker and concatenated with that worker's feature vector. This creates one task-worker row per possible action.

The shared encoder has two fully connected layers with ReLU activation:

$$\text{task-worker features} \rightarrow \text{Dense}(128) \rightarrow \text{ReLU} \rightarrow \text{Dense}(128) \rightarrow \text{ReLU}$$

The actor-critic data path is shown in Figure 3.7.1, rendered directly from the project Mermaid specification.

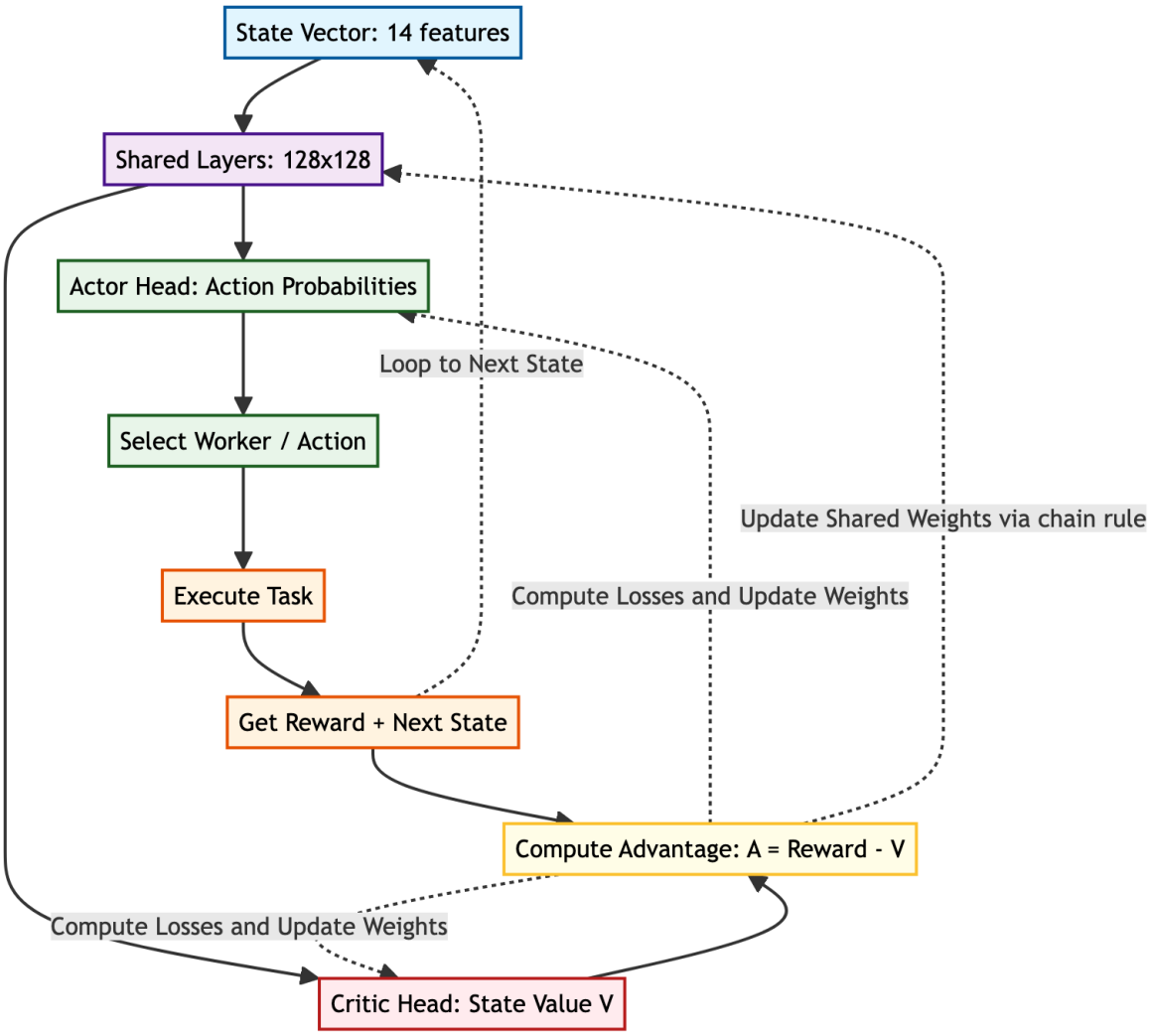

Figure 3.7.1: Actor-critic data flow and PPO learning feedback loop

After the shared encoder, the network splits into two heads:

- **Actor / policy head**: produces one logit per worker. After masking infeasible workers, these logits define the probability distribution $\pi_\theta(a|s)$.
- **Critic / value head**: produces a single value estimate $V_\theta(s)$, which predicts the expected future reward from the current cluster state.

The critic is not choosing the worker. Its job is to judge the state. During training, it answers the question: "From this task and current worker set, how much reward should the scheduler expect on average?" PPO then compares the actual reward to this expectation. If the selected action performs better than the critic expected, the advantage is positive and PPO increases the probability of similar choices. If it performs worse, the advantage is negative and PPO reduces that probability.

Formally, the critic estimates discounted return:

$$V_\theta(s_t) = \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t\right].$$

In this scheduler, $V_\theta(s)$ is not a probability and does not have a fixed $[0, 1]$ range. Its scale follows the reward design and discount horizon. Single-step rewards are usually around a bounded practical band (for example, infeasible placements receive -1.8, while feasible low-pressure placements can stay positive because of the $1.4 + 0.25H$ terms), but queue and tail-pressure penalties can drive rewards lower during congestion. Therefore $V_\theta(s)$ is typically positive in healthy, balanced states and can become negative under sustained pressure, repeated requeues, or poor placements. Magnitude increases with effective horizon ($\approx 1/(1-\gamma)$) because it represents cumulative discounted reward.

In implementation, the value head is applied after pooling the encoded worker representations. This means the critic sees the whole candidate set, not only the selected worker. That matters because a scheduling state is defined by all available workers together. A worker with 3 GB free memory is good or bad depending on what the other workers look like.

The critic is trained using a clipped value loss:

$$L^{VF} = \max\left((V_\theta - V_{target})^2,\ (V_{clipped} - V_{target})^2\right)$$

where $V_{clipped}$ limits how far the value estimate can move in one update. This mirrors PPO's conservative policy update. It prevents the critic from suddenly changing its estimate too much, which would make advantage estimates unstable.

The value target is taken from the rollout return, not from a fixed constant. Using standard PPO with GAE, we first compute

$$\delta_t = r_t + \gamma V_{\theta old}(s_{t+1}) - V_{\theta old}(s_t), \qquad \hat{A}_t = \sum_{l=0}^{T-1} (\gamma\lambda)^l \delta_{t+l}.$$

Then the critic target is

$$V_{target,t} = \hat{A}_t + V_{\theta old}(s_t),$$

which is the bootstrapped λ-return for that state. If a trajectory segment ends at a terminal state, the bootstrap term from $V_{\theta old}(s_{t+1})$ is dropped (set to zero).

The total PPO loss combines the policy loss, the value loss, and an entropy term:

$$L = L^{CLIP} + c_1 L^{VF} - c_2 H[\pi_\theta]$$

For a categorical policy over workers, entropy is:

$$H[\pi_\theta(\cdot|s)] = -\sum_{a=1}^{N_w} \pi_\theta(a|s) \log \pi_\theta(a|s),$$

where N_w is the number of feasible worker actions after masking and $\pi_\theta(a|s)$ comes from the masked softmax probabilities. High entropy means probabilities are spread across

workers (exploration), while low entropy means the policy is concentrated on a small set of workers (near-deterministic behavior). In this project, the value coefficient c_1 weights critic learning, and the entropy coefficient c_2 prevents premature determinism where the scheduler repeatedly selects one apparently strong worker too early. As training progresses and the model becomes better calibrated, entropy naturally decreases.

3.8 | State Representation

The PPO state is built from task and worker features. The task vector has five values:

Table 3.8.1: Task feature vector

Feature	Meaning
Required CPU	CPU requested by the task.
Required memory	Memory requested by the task.
Required storage	Storage requested by the task.
SLA multiplier	How much delay is acceptable relative to task runtime.
Task type	Encoded task category such as CPU-heavy, memory-heavy, or mixed.

Each worker is represented using nine values:

Table 3.8.2: Worker feature vector

Feature	Meaning
Available CPU ratio	Available CPU divided by total CPU.
Available memory ratio	Available memory divided by total memory.
Available storage ratio	Available storage divided by total storage.
Total CPU	Absolute CPU capacity.
Total memory	Absolute memory capacity.
Total storage	Absolute storage capacity.
Used CPU ratio	Used CPU divided by total CPU.
Used memory ratio	Used memory divided by total memory.
Used storage ratio	Used storage divided by total storage.

The model uses pairwise task-worker features. This is important because the question is not only “what is the task?” or “what is the worker?” but “how suitable is this worker for this task?”

3.9 | Action Space and Masking

The action is the selected worker index. Before PPO chooses, the system builds an action mask. A worker is marked feasible only if it is active and has enough CPU, memory, and

storage for the task. Infeasible workers are masked so the model cannot choose them during normal operation.

3.10 | Reward Function

The reward was designed to make PPO learn practical scheduling behavior:

$$R = 1.4 + 0.25H - 0.35Q_p - 0.55T_p - 0.20I_p - 0.40\Delta I - R_q$$

where:

- H is headroom bonus. It rewards choosing workers with space left.
- Q_p is queue pressure. It penalizes expected waiting.
- T_p is tail pressure. It penalizes likely SLA or tail-latency risk.
- I_p is imbalance penalty. It discourages putting more work on already busy workers.
- ΔI is change in imbalance. It penalizes actions that make cluster balance worse.
- R_q is requeue penalty. It discourages repeatedly risky placements.

The values are computed from the projected state after placing the task on the selected worker. First, the system estimates worker load as the average of CPU, memory, and storage usage:

$$L_w = \frac{usedCPU_w/totalCPU_w + usedMem_w/totalMem_w + usedStorage_w/totalStorage_w}{3}$$

After adding the new task's requested resources to the selected worker, this becomes the projected load $L_{selected}$. The cluster average load is:

$$L_{cluster} = \frac{1}{N}\sum_{w=1}^{N} L_w$$

Using these values, the reward terms are:

Table 3.10.1: Reward term computation

Term	How it is computed and what it means
H	$1 - L_{selected}$. A lightly loaded selected worker gives a higher headroom bonus. If the worker is almost full, this value becomes small.
Q_p	$\min(queueWaitProxy/slaBudget, 3.0)$. The proxy is trace queue wait plus $runtime \times L_{selected}$. It estimates how much waiting pressure the placement creates relative to the SLA budget.
T_p	$\max(turnaroundProxy/slaBudget - 1, 0)$. Turnaround proxy is queue wait proxy plus runtime. This term becomes positive only when expected turnaround crosses the SLA budget.
I_p	$\max(L_{selected} - L_{cluster}, 0)$. This is imbalance in the system. It means the selected worker is more loaded than the cluster average. If the selected worker is below or equal to average load, the penalty is zero.
ΔI	$\sigma(loads_{after}) - \sigma(loads_{before})$. This measures change in imbalance using the standard deviation of worker loads. Positive means the action made the cluster less balanced. Negative means it made the cluster more balanced.
R_q	$0.05 \times \min(requeueCount, 4)$. A task that has already been requeued receives a small extra penalty so the scheduler avoids repeating risky placements.

For example, if one worker is already much busier than the others, its L_w will be above $L_{cluster}$. Placing another task there increases I_p, and it may also increase ΔI if the spread between worker loads becomes larger. In system terms, imbalance means a hot-spot: one worker is carrying more active resource load while other workers still have useful capacity. The reward therefore teaches PPO to avoid placing every task on the same apparently strong worker.

If a worker is infeasible, the reward is -1.8. This strong negative reward teaches the policy that violating hard resource limits is worse than merely making a slow but feasible placement.

3.11 | Training Data

The PPO model was trained using trace replay. The Alibaba cluster trace was used because it contains real production workload behavior [5]. The trace replay environment presents tasks in chronological order. When a task is placed, its resources are reserved on the selected worker. When its simulated runtime ends, resources are released.

Decision: Lifecycle-based resource tracking was used instead of simple exponential decay. This matters because many trace arrivals are simultaneous. A decay model can release or retain resources incorrectly, while lifecycle tracking matches the actual running period of each simulated task.

3.12 | Training Pipeline

The training pipeline follows these steps:

1. Load trace tasks and worker information.
2. Normalize task and worker features.
3. Create the replay environment.
4. Run PPO rollouts and collect actions, rewards, log probabilities, and values.
5. Compute returns and advantages.
6. Update the policy using PPO clipping, value loss, entropy regularization, and gradient clipping.
7. Save model checkpoints.
8. Promote the best checkpoint for cluster use.

3.13 | Implementation in the Cluster

The Go master has a PPO scheduler wrapper. For each task, it builds a candidate worker list and sends it to the Python PPO service. The service encodes the request, applies the action mask, runs the policy, and returns the selected worker ID.

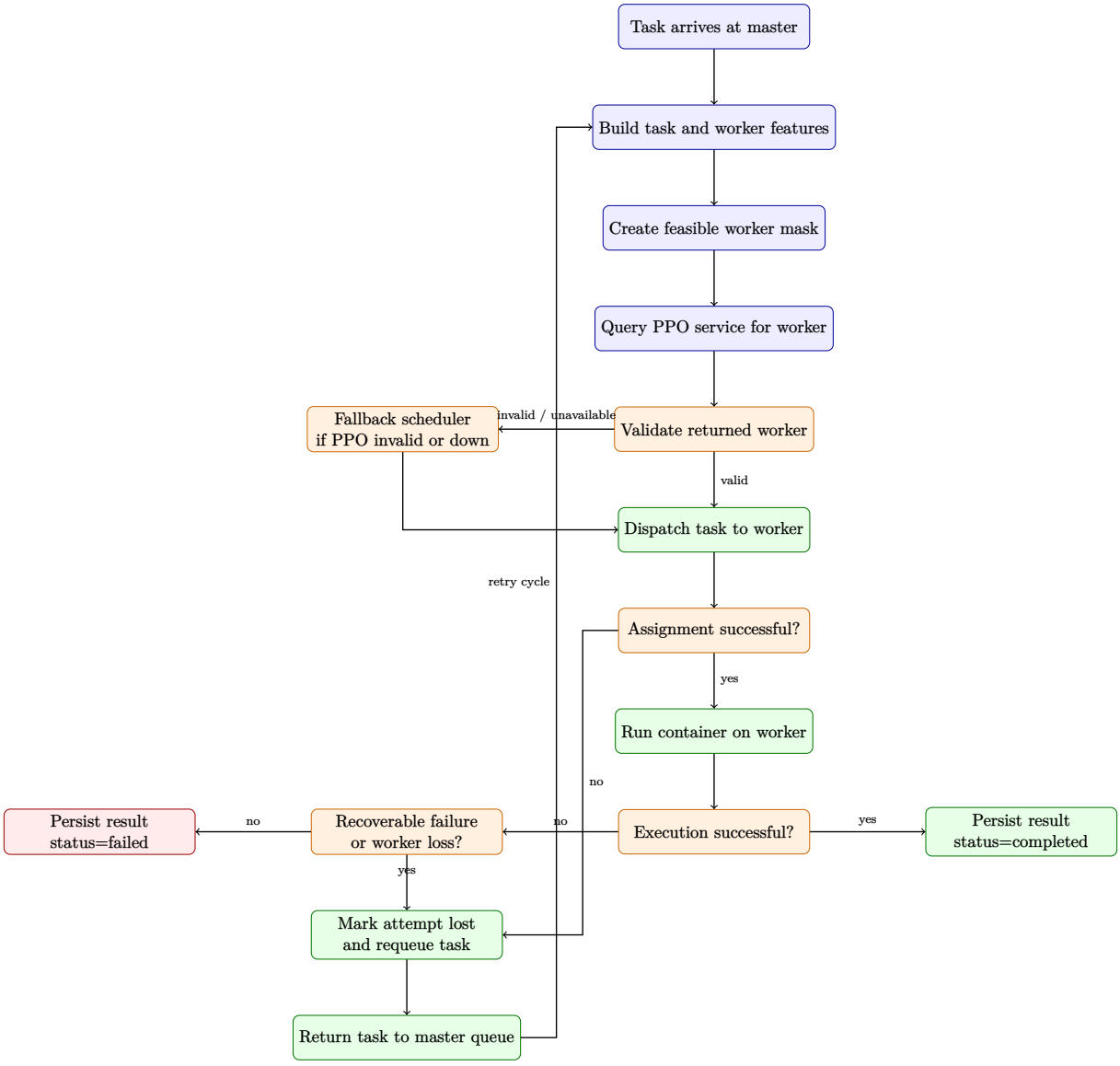

Figure 3.13.1: PPO scheduling flow inside the Go cluster with failover and requeue

Explanation of Figure 3.13.1: The flow starts in the Go master, not inside the neural network. After PPO returns a worker, the master validates that worker against the latest state. If PPO is unavailable or the choice is invalid, fallback scheduling is used. Assignment failures and recoverable runtime failures are not terminal: the current attempt is marked lost and the task is requeued for another scheduling cycle. Only non-recoverable execution failures are finalized as failed.

The implementation also includes fallback behavior:

- If the PPO service is down, use the fallback scheduler.
- If PPO returns an empty result, use fallback.
- If PPO returns a worker that no longer exists or cannot fit the task, use fallback.
- If shadow mode is enabled, log PPO's decision but execute fallback's decision.

Decision: The learning model is never allowed to be the only safety mechanism. The Go scheduler validates PPO's output before assigning a task.

Chapter 4

Results

4.1 | Project Outcomes

This chapter reports the measurable outcomes of the project and ties those outcomes to practical system behavior in the deployed cluster.

4.1.1 | Evaluation Setup

The system was evaluated in two ways:

1. **Offline PPO evaluation**: the trained model was tested on trace replay data to measure reward and feasibility.
2. **End-to-end cluster campaigns**: the Go master and workers executed real Docker tasks under different workload patterns and scheduler choices.

The main schedulers compared were:

Table 4.1.1: Schedulers compared

Scheduler	Description
Round-Robin	Simple cyclic assignment baseline.
RTS	Risk-aware heuristic scheduler using resource and risk scoring.
PPO	Reinforcement learning scheduler trained through trace replay.

4.1.2 | Training Convergence

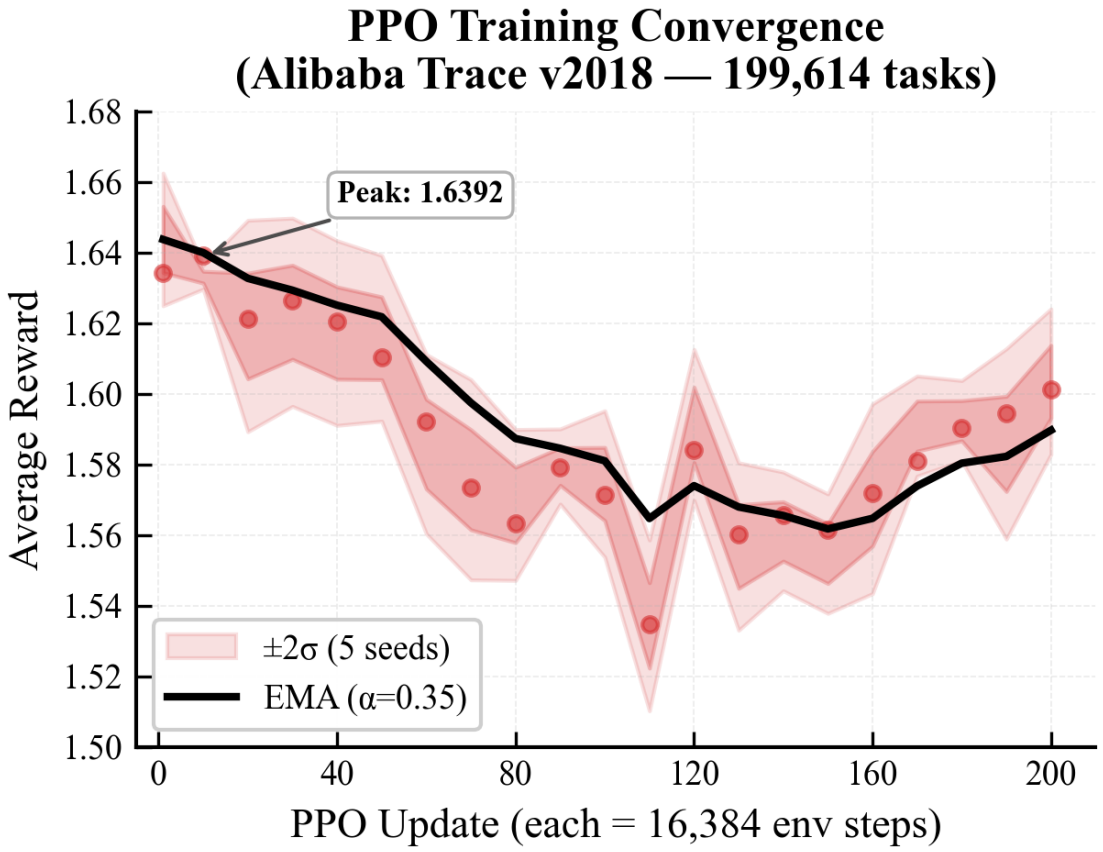

Figure 4.1.1: PPO training reward curve

Explanation of Figure 4.1.1: The curve shows how the average reward changed during training. Early training is exploratory because the policy is still testing placements. Later, the reward stabilizes, which indicates that the policy has learned a consistent placement strategy for the replayed cluster trace.

Average reward means the mean reward received per scheduling decision during a training window. From the DRL point of view, it is the signal that tells whether the actor is choosing actions that the environment considers better than before. A rising average reward means the policy is getting more positive feedback from the reward function. A flat reward means the policy has probably reached a stable behavior for the current trace and hyperparameters.

From the scheduling point of view, average reward is not a raw time measurement. It is a combined score for practical placement quality: feasible placement, enough worker headroom, lower waiting pressure, lower tail-risk pressure, and better cluster balance. So when PPO's average reward improves, it means the scheduler is learning to make placements that preserve resources and avoid risky worker choices, not only placements that finish one task quickly.

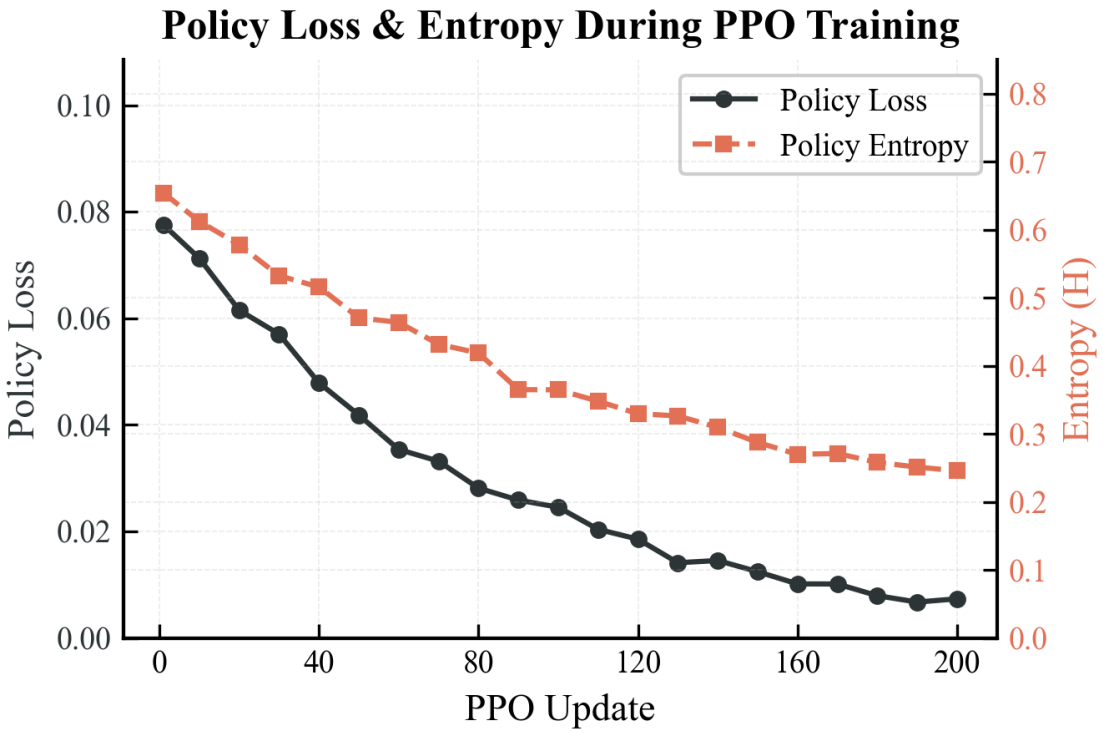

Figure 4.1.2: PPO policy loss and entropy

Explanation of Figure 4.1.2: This figure shows policy loss and entropy. Policy loss is the PPO actor loss. It measures how much the policy is still changing to make high-advantage actions more likely and low-advantage actions less likely. In simple words, it shows how much correction the actor still needs. A decreasing policy loss means the updates are becoming smaller because the scheduler is learning a more stable placement rule.

Entropy measures randomness in the policy distribution. From the DRL point of view, high entropy means the agent is exploring many actions; low entropy means it is confident and more deterministic. From the scheduling point of view, high entropy means PPO is still trying different workers for similar tasks. Lower entropy means it has learned stronger preferences, for example preferring a worker with better resource fit and lower tail risk.

Value loss is not drawn in this figure because the chart was kept to two axes for readability, but it is still part of the PPO update. Value loss belongs to the critic network. It measures the error between the critic's predicted state value $V(s)$ and the actual return observed from the rollout. From the scheduling point of view, value loss tells whether the critic is learning to judge cluster states correctly. If value loss falls, the critic is becoming better at estimating whether the current task-worker situation will lead to good future scheduling rewards.

4.1.3 | Offline PPO Model Results

The word trace in this section means a historical workload record. It contains task arrivals, requested resources, runtimes, and machine information. During trace replay, the environment feeds these tasks to the scheduler in chronological order. The scheduler does not simply copy the historical placement; it chooses workers in the simulated cluster,

reserves resources for the simulated runtime, releases resources when tasks finish, and receives reward for each decision.

The best PPO model achieved a mean reward of approximately 0.8801 on the evaluation trace, compared with 0.8688 for Round-Robin. The feasible action rate also improved from 94.38% for Round-Robin to 94.86% for PPO.

Table 4.1.2: Offline scheduler comparison on trace replay

Policy	Mean Reward	Feasible Action Rate
Round-Robin	0.8688	94.38%
First feasible	0.8145	94.80%
Max available	0.8800	94.84%
PPO best model	**0.8801**	**94.86%**

Mean reward is calculated by averaging the reward values over all scheduling decisions in the evaluation trace. It is based on the same reward function described in the methodology: feasibility, headroom, queue pressure, tail pressure, imbalance, change in imbalance, and requeue penalty. Feasible action rate is calculated as:

$$\text{Feasible Action Rate} = \frac{\text{number of selected workers that satisfy task resources}}{\text{total scheduling decisions}} \times 100$$

In scheduling terms, feasible action rate measures how often the scheduler chose a worker that could actually run the task without violating CPU, memory, or storage constraints. The improvement is not huge in light-load trace replay, but it is meaningful because PPO reached the level of a strong heuristic while keeping the ability to learn from reward signals.

4.1.4 | End-to-End KPI Comparison

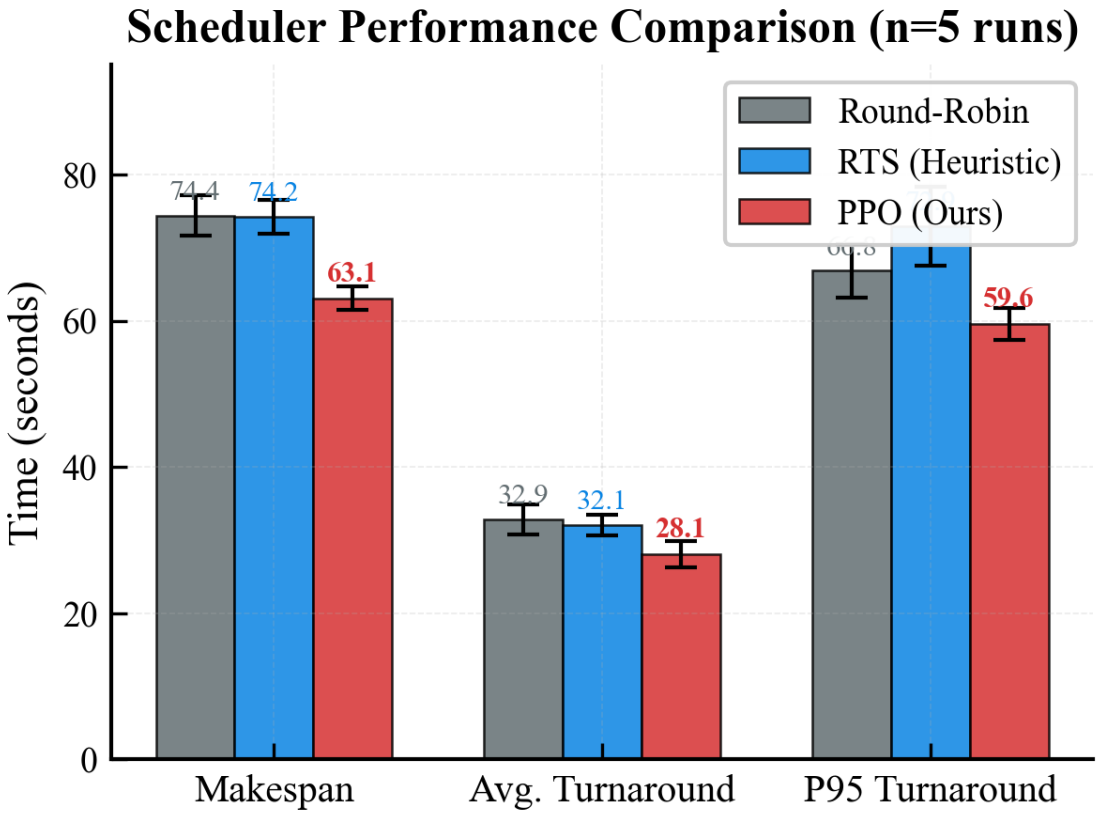

Figure 4.1.3: KPI comparison between schedulers

Explanation of Figure 4.1.3: This plot compares the major scheduler metrics. The important reading is not that PPO wins every metric in every case. The honest result is that PPO becomes most valuable under pressure, while simpler schedulers can remain competitive when the workload is easy. This is a realistic systems result because adding intelligence also adds inference overhead.

4.1.5 | Multi-Run Comparison

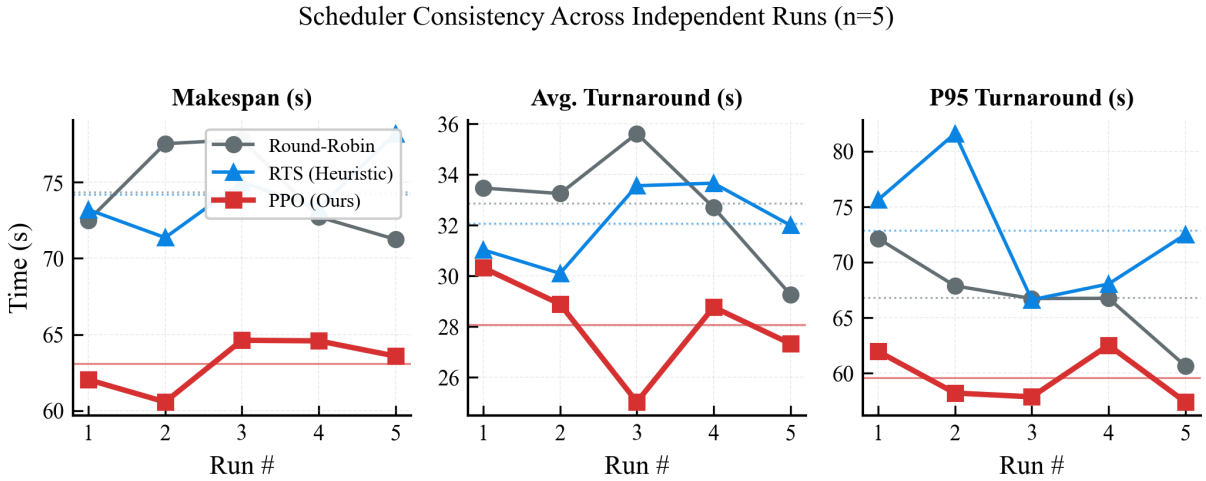

Figure 4.1.4: Multi-run scheduler comparison

Explanation of Figure 4.1.4: A single run can be misleading because task timing, container startup, and worker state can vary. The multi-run plot compares schedulers across repeated runs. It shows whether a scheduler is consistently better or only lucky in one campaign. This helped identify that PPO's advantage is clearer in burst and overload conditions than in simple steady cases.

4.1.6 | Cumulative Completion

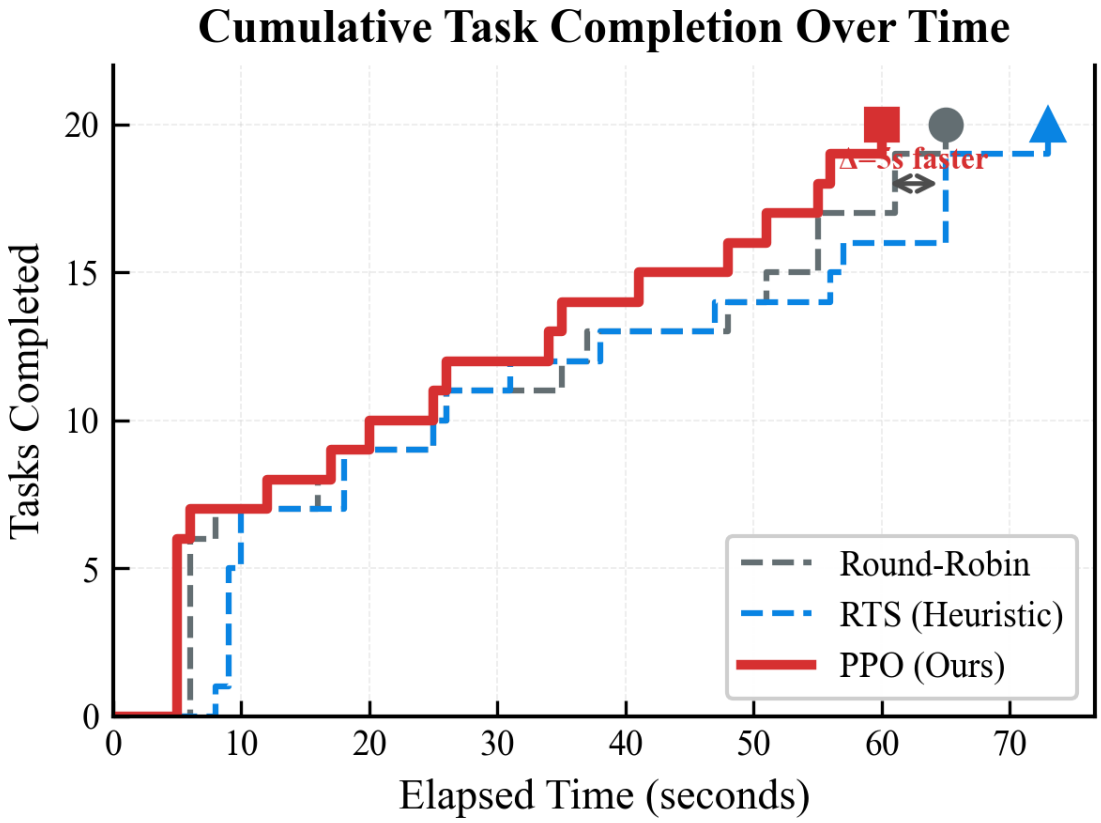

Figure 4.1.5: Cumulative task completion over time

Explanation of Figure 4.1.5: This plot shows how quickly each scheduler finishes tasks over time. A steeper curve means tasks are completing faster. This view is useful because average duration alone can hide long-tail behavior. In cluster scheduling, tail behavior matters because a few delayed tasks can block an entire workload.

4.1.7 | WebUI Operational Visuals

During live campaigns, the WebUI was used as an operator-facing view to validate that scheduler decisions matched observed system behavior. It complements the numeric KPI tables by showing cluster state and workload progress in real time.

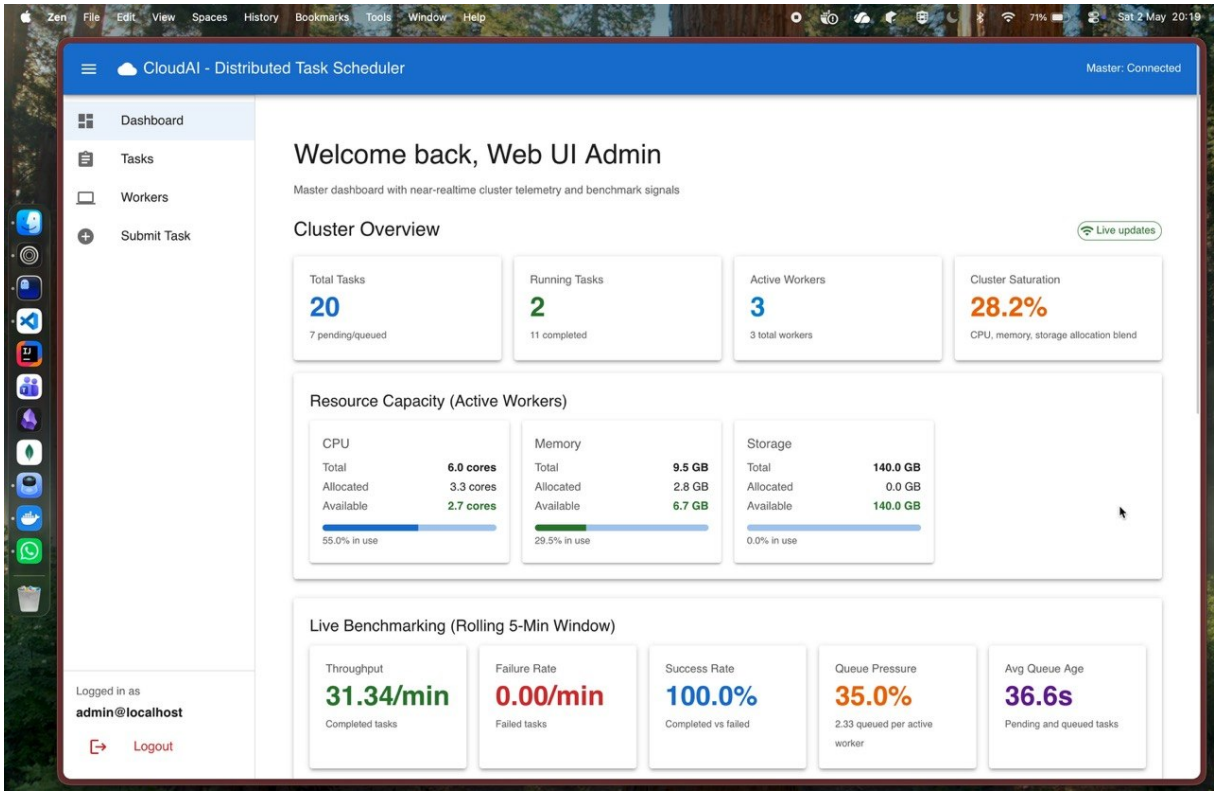

Figure 4.1.6: WebUI dashboard view during campaign execution

Explanation of Figure 4.1.6: The dashboard view provides a consolidated status of the active scheduler, worker availability, and task lifecycle progression. This helped verify that PPO placement decisions were not only better in aggregate metrics, but also operationally consistent while the workload was running.

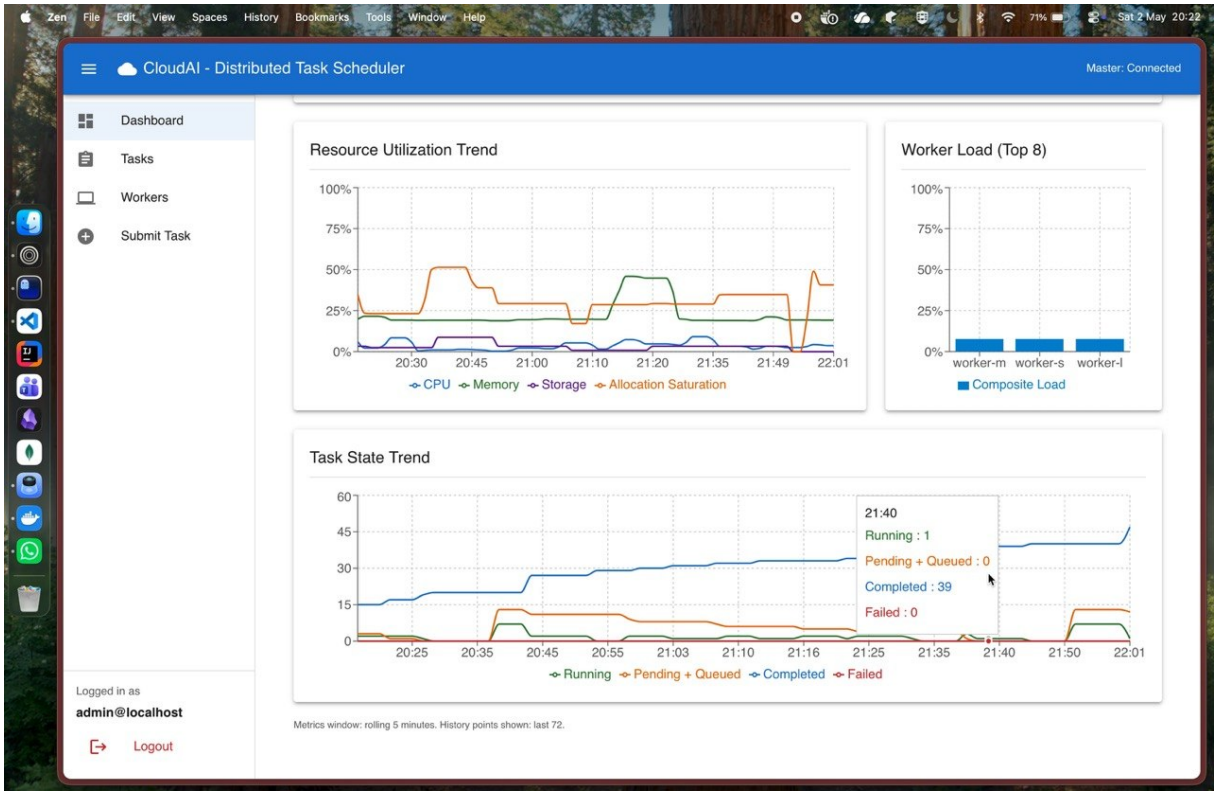

Figure 4.1.7: WebUI trend plot for scheduler performance tracking

Explanation of Figure 4.1.7: The trend plot visualizes runtime behavior across the campaign window, making it easier to inspect response under pressure and detect tail effects that may be hidden in simple averages. This visual evidence aligns with the measured improvements in turnaround and P95 completion for PPO in contention-heavy scenarios.

4.1.8 | Latest Campaign Summary

The campaign testbench is the end-to-end evaluation harness for the project. It runs against the live Go master over HTTP, switches the active scheduler, submits a fixed workload, polls task status until tasks become terminal or the timeout is reached, and then records duration, success rate, queue wait, average turnaround, and P95 turnaround. This is different from offline trace replay because the tasks are real Docker containers running on registered workers.

This campaign ran the `resource-contention-ppo` workload with 20 tasks per scheduler. All schedulers achieved 100% success, but PPO had the best overall runtime behavior. PPO finished the campaign in 63.95 seconds, compared with 70.06 seconds for Round-Robin and 76.20 seconds for RTS. PPO also had the lowest average turnaround and lowest P95 turnaround.

Table 4.1.3: New-model campaign scheduler summary

Scheduler	Tasks	Completed	Failed	Duration	Turnaround	P95
RR	20	20	0	70.06 s	33.80 s	69.00 s
RTS	20	20	0	76.20 s	33.40 s	74.00 s
PPO	20	20	0	**63.95 s**	**27.75 s**	**62.00 s**

Table 4.1.4: New PPO improvement over baselines

Comparison	Duration Improvement	Turnaround Improvement
PPO vs RR	8.7% faster	17.9% lower
PPO vs RTS	16.1% faster	16.9% lower

The difference came from execution placement: PPO selected workers in a way that reduced total campaign duration and reduced the tail turnaround. Turnaround time means the time from task creation to task completion. Tail turnaround focuses on the slowest part of the workload, usually measured using a high percentile such as P95. In this campaign, P95 turnaround means that almost all tasks completed within that many seconds, except the slowest few tasks. PPO reducing P95 from 69.00 seconds for Round-Robin and 74.00 seconds for RTS to 62.00 seconds means the slowest tasks were less delayed. This is important because a cluster can look acceptable on average while still having a few tasks that finish very late.

4.1.9 | Pressure Scenarios

The pressure tests were designed to make scheduling non-trivial. The testbench can run baseline, burst, and overload modes. In baseline mode, one copy of the workload is submitted and the runner waits up to 600 seconds for completion. In burst mode, the same workload is submitted without spacing between tasks, again with a 600 second per-scenario timeout. In overload mode, the workload is repeated three times to saturate the cluster, and the timeout is increased to 1200 seconds.

The `resource-contention-ppo` workload used in the final campaign contains 20 deterministic Docker tasks. The task set is mixed on purpose: five large CPU-heavy tasks, five memory-heavy tasks, five medium mixed CPU-memory tasks, and five small CPU-light tasks. The resource requests are also different. Large tasks request 2.5 CPU and 0.8 GB memory, memory-heavy tasks request 0.8 CPU and 2.0 GB memory, medium tasks request 1.5 CPU and 0.8 GB memory, and small tasks request 0.5 CPU and 0.4 GB memory. This creates a bin-packing problem because not every task is equally suitable for every worker.

Earlier comprehensive campaigns showed the clearest PPO advantage under burst load. In one campaign, Round-Robin and RTS timed out on multiple burst workloads, while PPO completed a large percentage of burst tasks. This supports the original motivation: learned scheduling matters most when the placement decision is non-obvious.

Table 4.1.5: Burst scenario interpretation

Observation	Interpretation
RR performs well under light load	Simple cyclic placement is enough when all workers have spare capacity.
RTS improves some resource-aware cases	Heuristics help when they match the workload pattern.
PPO improves under burst or overload pressure	Learned placement can capture interactions between task size, worker state, and tail risk.
PPO has overhead	A learned scheduler must justify its cost by improving difficult cases.

4.1.10 | Important Engineering Decisions and Results

Table 4.1.6: Key project decisions and outcomes

Decision	Reason	Outcome
Use Go for master and worker	The control plane needs efficient networking and concurrency.	The cluster can run master-worker task execution with clear service boundaries.
Use Docker for task execution	Tasks need portable execution environments.	Workers can run containerized workloads with resource requirements.
Use action masking in PPO	Invalid workers must not be selected.	PPO decisions respect hard resource constraints.
Use lifecycle resource tracking in trace replay	Decay-based resource release was inaccurate for simultaneous arrivals.	Training state better matches real task occupation.
Add tail-pressure reward	Average completion is not enough; long-tail delays matter.	PPO is guided toward reducing risky placements.
Use fallback scheduler validation	A learned model can fail or be unavailable.	Cluster safety is preserved even when PPO cannot decide.

4.1.11 | Delivered Components

The completed project delivers:

- a working distributed task execution cluster,
- worker registration and heartbeat tracking,
- task assignment and result reporting,

- scheduler comparison infrastructure,
- PPO trace replay training,
- PPO service integration with Go,
- offline and end-to-end evaluation artifacts.

4.2 | My Contributions to the Project

The main individual contributions were the design and development of the Go cluster framework, the scheduler integration boundary, the PPO scheduling pipeline, the training and evaluation workflow, and the final benchmark interpretation. A major part of the work was also debugging practical system issues: resource accounting, worker visibility, fallback behavior, model loading, and repeatable campaign execution.

4.3 | Learning Outcomes

The project helped connect two areas that are often studied separately: distributed systems and reinforcement learning. The distributed systems part taught how much correctness depends on state handling, retries, and clean boundaries. The RL part taught that model quality depends on reward design, feature representation, and safe deployment. The most important lesson was that learning-based scheduling should be added as a controlled improvement, not as a replacement for basic system reliability.

4.4 | Real World Applications

This project applies to small cloud platforms, internal batch execution systems, university lab clusters, CI workload runners, and edge clusters where machines have different capacities. The PPO scheduler is especially relevant when workloads are mixed and manual scheduling rules become hard to tune.

In a university or research lab, the framework can be used to share a few machines among many users running experiments. Some experiments may be CPU-heavy, some may be memory-heavy, and some may be short utility jobs. A learned scheduler can reduce the chance that a small job is blocked behind a badly placed large job.

In CI and testing systems, the framework can schedule build, test, and benchmark jobs across workers with different capacities. A simple Round-Robin scheduler may send a heavy integration test to a weak machine, while PPO can learn from previous outcomes and prefer better placements.

In edge computing, machines are often heterogeneous and resource-limited. A scheduler must decide where to run tasks without assuming every node has the same capacity. The same master-worker idea can be used to route video processing, data filtering, or inference jobs to suitable edge nodes.

In private cloud environments, the project can act as a lightweight batch execution layer. It is not intended to replace large orchestration platforms, but it shows how a focused scheduler can be built, tested, and improved with reinforcement learning while still keeping safe fallback behavior.

Chapter 5

Conclusion

Agentic Cloud Cluster successfully demonstrates a Go-based distributed task execution framework with PPO-based scheduling. The project began with the practical need to assign containerized tasks across heterogeneous workers and developed into a complete scheduling study. The final design separates the cluster control plane from the learning service, which keeps the system maintainable and safe.

The methodology showed why reinforcement learning is a reasonable fit for cluster scheduling. Q-tables explain the basic idea but fail because the state space is too large. DQN improves generalization but is less natural for dynamic worker sets and action masking. PPO was chosen because it directly learns a worker-selection policy, supports stable clipped updates, and works well with continuous resource features.

The results show a balanced picture. PPO is not automatically better in every scenario. When load is light, simple schedulers are fast and effective. Under burst and overload pressure, PPO becomes more useful because it can use learned relationships between task demand, worker state, and risk. This is the main conclusion of the project: learning-based scheduling is valuable when the decision space is complex, but it must be deployed with validation and fallback.

5.1 | Future Work

Future work can improve the project in the following ways:

- Train on larger and more diverse traces.
- Add multi-objective reward tuning for cost, energy, and fairness.
- Improve online adaptation after longer stable cluster runs.
- Evaluate with more physical or cloud machines.
- Add stronger failure injection tests for worker loss and network delay.
- Explore CRIU-based container checkpoint/restore to support faster recovery and future preemption or migration workflows.

Overall, the project proves that a distributed systems framework and a reinforcement learning scheduler can be combined in a practical and safe way.

Bibliography

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2 ed., 2018.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[5] Alibaba, "Alibaba cluster data." https://github.com/alibaba/clusterdata, 2018. Cluster traces released for cluster management research.

[6] Docker Inc., "Docker documentation." https://docs.docker.com/, 2026. Accessed May 2026.

[7] The Go Authors, "Documentation - the go programming language." https://go.dev/doc/, 2026. Accessed May 2026.

[8] gRPC Authors, "Introduction to grpc." https://grpc.io/docs/what-is-grpc/introduction/, 2026. Accessed May 2026.

[9] MongoDB, "Introduction to mongodb." https://www.mongodb.com/docs/manual/introduction/, 2026. Accessed May 2026.

Appendix

.1 | Repository Modules

Table .1.1: Main repository modules used in the project

Path	Purpose
`master/`	Go master node, scheduler interface, task handling, worker handling, persistence hooks.
`worker/`	Go worker node, Docker task execution, heartbeat reporting, log streaming.
`proto/`	Protocol Buffer definitions for master-worker and PPO scheduler communication.
`agentic_scheduler/`	Python PPO model, service, training code, trace replay environment, checkpoints.
`testbench/`	Workload and campaign automation for end-to-end evaluation.
`results/`	Benchmark reports, campaign outputs, plots, and extracted result summaries.

.2 | Representative PPO Configuration

Table .2.1: Representative PPO training configuration

Parameter	Value / Meaning
Model	Actor-critic neural network with two hidden layers.
Hidden size	128.
Task feature dimension	5.
Worker feature dimension	9.
Clip ratio	0.2.
Value coefficient	0.5.
Entropy coefficient	Tuned during experiments, commonly around 0.01–0.02.
Training source	Alibaba trace replay and project campaign traces.

.3 | Ports and Service Mapping

Table .3.1: Ports used by the project

Component	Port / Address	Purpose
Master gRPC server	`0.0.0.0:50051`	Main worker-to-master communication endpoint for registration, task assignment coordination, heartbeats, and result reporting.
Master HTTP API	`0.0.0.0:8080`	HTTP API used for task, worker, file, auth, health, and metric endpoints.
PPO scheduler service	`127.0.0.1:50050`	Python gRPC service used by the Go PPO scheduler for `Ping`, model loading, worker selection, and outcome reporting.
Worker 1 gRPC	`0.0.0.0:50052`	Worker task execution endpoint.
Worker 2 gRPC	`0.0.0.0:50053`	Worker task execution endpoint.
Worker 3 gRPC	`0.0.0.0:50054`	Worker task execution endpoint.
Additional workers	`0.0.0.0:5005N`	Extra worker nodes can use the same pattern with unique ports.
MongoDB	`localhost:27017`	Persistent storage for tasks, workers, assignments, attempts, results, users, and scheduler model metadata.
Worker metrics	`0.0.0.0:9101+`	Per-worker metrics endpoint. Each worker can use a unique metrics port when multiple workers run on the same host.
Docker daemon	Host socket / Docker API	Container execution backend used by worker nodes. Exact access path depends on local Docker setup.

These mappings are the default local development mappings used by the project. In a multi-machine deployment, the same logical services remain, but the hostnames change from `localhost` to routable machine addresses.